



BUBT | BANGLADESH UNIVERSITY OF
BUSINESS AND TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

LAB REPORT

FINAL

Compiler Design Lab (CSE-324)

Submitted By:

Kaniz Fatema

20245103154

Intake 53 (03)

Submitted To:

Md. Nahid Hossain

Lecturer

Dept. of CSE

August 17, 2026

Contents

1	LL(1) parsing	3
1.0.1	Question	3
1.0.2	Solution	3
2	Recursive Descent Parsing	10
2.0.1	Question	10
2.0.2	Solution	10
3	Predictive parsing table	14
3.0.1	Question	14
3.0.2	Solution	14
4	Remove single Line comment	20
4.0.1	Question	20
4.0.2	Solution	20
5	Three address code	22
5.0.1	Question	22
5.0.2	Solution	22

1 LL(1) parsing

1.0.1 Question

1. Write a program to implement LL(1) parsing for the following grammar

1.0.2 Solution

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 map<char, vector<string>> G;
5 map<char, set<char>> FIRST, FOLLOW;
6 map<char, map<char, string>> table;
7
8 char startSymbol;
9
10 bool isNT(char c)
11 {
12     return c >= 'A' && c <= 'Z';
13 }
14
15 set<char> firstString(string s)
16 {
17     set<char> ans;
18
19     if (s == "#")
20     {
21         ans.insert('#');
22         return ans;
23     }
24
25     for (char c : s)
26     {
27         if (!isNT(c))
28         {
29             ans.insert(c);
30             return ans;
31         }
32
33         for (char x : FIRST[c])
34             if (x != '#')
35                 ans.insert(x);
36
37         if (!FIRST[c].count('#'))
38             return ans;
39     }
40
41     ans.insert('#');
42     return ans;
```

```

43 }
44
45 void findFirst()
46 {
47     bool change = true;
48
49     while (change)
50     {
51         change = false;
52
53         for (auto p : G)
54         {
55             char A = p.first;
56
57             for (string rhs : p.second)
58             {
59                 set<char> f = firstString(rhs);
60
61                 for (char x : f)
62                     if (FIRST[A].insert(x).second)
63                         change = true;
64             }
65         }
66     }
67 }
68
69 void findFollow()
70 {
71     FOLLOW[startSymbol].insert('$');
72
73     bool change = true;
74
75     while (change)
76     {
77         change = false;
78
79         for (auto p : G)
80         {
81             char A = p.first;
82
83             for (string rhs : p.second)
84             {
85                 for (int i = 0; i < rhs.size(); i++)
86                 {
87                     char B = rhs[i];
88
89                     if (!isNT(B))
90                         continue;
91
92                     string beta = rhs.substr(i + 1);

```

```

93
94         if (beta != "")
95         {
96             set<char> f = firstString(beta);
97
98             for (char x : f)
99             {
100                 if (x != '#')
101                 {
102                     if (FOLLOW[B].insert(x).second)
103                         change = true;
104                 }
105             }
106
107             if (f.count('#'))
108             {
109                 for (char x : FOLLOW[A])
110                     if (FOLLOW[B].insert(x).second)
111                         change = true;
112             }
113         }
114         else
115         {
116             for (char x : FOLLOW[A])
117                 if (FOLLOW[B].insert(x).second)
118                     change = true;
119         }
120     }
121 }
122 }
123 }
124 }
125
126 void makeTable()
127 {
128     for (auto p : G)
129     {
130         char A = p.first;
131
132         for (string rhs : p.second)
133         {
134             set<char> f = firstString(rhs);
135
136             for (char x : f)
137                 if (x != '#')
138                     table[A][x] = rhs;
139
140             if (f.count('#'))
141                 for (char x : FOLLOW[A])
142                     table[A][x] = rhs;

```

```

143     }
144 }
145 }
146
147 string showStack(stack<char> s)
148 {
149     string x = "";
150
151     while (!s.empty())
152     {
153         x += s.top();
154         s.pop();
155     }
156
157     return x;
158 }
159
160 void parse(string input)
161 {
162     input += '$';
163
164     stack<char> st;
165     st.push('$');
166     st.push(startSymbol);
167
168     int i = 0;
169
170     cout << "\nStack\tInput\tAction\n";
171     cout << "-----\n";
172
173     while (!st.empty())
174     {
175         char top = st.top();
176         char current = input[i];
177
178         cout << showStack(st) << "\t"
179              << input.substr(i) << "\t";
180
181         if (top == current)
182         {
183             if (top == '$')
184             {
185                 cout << "Accepted\n";
186                 cout << "\nString is accepted\n";
187                 return;
188             }
189
190             cout << "Match " << current << endl;
191
192             st.pop();

```

```

193         i++;
194     }
195
196     else if (isNT(top))
197     {
198         if (!table[top].count(current))
199         {
200             cout << "No Rule\n";
201             cout << "\nString is not accepted\n";
202             return;
203         }
204
205         string rhs = table[top][current];
206
207         cout << top << "->" << rhs << endl;
208
209         st.pop();
210
211         if (rhs != "#")
212         {
213             for (int j = rhs.size() - 1; j >= 0; j--)
214                 st.push(rhs[j]);
215         }
216     }
217
218     else
219     {
220         cout << "Mismatch\n";
221         cout << "\nString is not accepted\n";
222         return;
223     }
224 }
225 }
226
227 int main()
228 {
229     int n;
230
231     cout << "Enter number of productions: ";
232     cin >> n;
233
234     for (int i = 0; i < n; i++)
235     {
236         string p;
237         cin >> p;
238
239         char lhs = p[0];
240         string rhs = p.substr(3);
241
242         G[lhs].push_back(rhs);

```

```
243
244     if (i == 0)
245         startSymbol = lhs;
246     }
247
248     findFirst();
249     findFollow();
250     makeTable();
251
252     string input;
253
254     cout << "Enter string: ";
255     cin >> input;
256
257     parse(input);
258
259     return 0;
260 }
```

Listing 1: C++ program to LL(1) parsing.

Output

```
Problems 2 Output Debug Console Terminal Ports

Active code page: 65001

D:\code2026\cse324>cd "d:\code2026\cse324\lab-06-ll1-parsing\" && g++ ll1-parsing-any
Enter number of productions: 3
A->aCa
C->bC
C->#
Enter string: abba

Stack   Input   Action
-----
A$      abba$   A->aCa
aCa$    abba$   Match a
Ca$      bba$    C->bC
bCa$    bba$    Match b
Ca$      ba$     C->bC
bCa$    ba$     Match b
Ca$      a$      C->#
a$       a$      Match a
$        $       Accepted

String is accepted

d:\code2026\cse324\lab-06-ll1-parsing>
```

Figure 1: Output of LL(1) parsing

2 Recursive Descent Parsing

2.0.1 Question

1. Write a program to implement Recursive Descent Parsing for the following grammar.

Write a program to implement Recursive Descent Parsing for the following grammar

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid a$

Sample input: $a+a*a$

Sample output: String is accepted

Sample input: $a/a*a$

Sample output: String is not accepted

2.0.2 Solution

```
1 #include <iostream>
2 #include <map>
3 #include <vector>
4 #include <string>
5
6 using namespace std;
7
8 map<char, vector<string>> grammar;
9 string input;
10
11 // match a production
12 bool match(string rule, int pos, int rulePos, int &endPos);
13
```

```

14 // Try a non-terminal
15 bool parse(char nt, int pos, int &endPos)
16 {
17     for (string rule : grammar[nt])
18     {
19         int tempPos;
20
21         // Epsilon
22         if (rule == "#")
23         {
24             endPos = pos;
25             return true;
26         }
27
28         if (match(rule, pos, 0, tempPos))
29         {
30             endPos = tempPos;
31             return true;
32         }
33     }
34
35     return false;
36 }
37
38 // Match symbols of a production
39 bool match(string rule, int pos, int rulePos, int &endPos)
40 {
41     // Production finished
42     if (rulePos == rule.length())
43     {
44         endPos = pos;
45         return true;
46     }
47
48     char symbol = rule[rulePos];
49
50     // Non-terminal
51     if (symbol >= 'A' && symbol <= 'Z')
52     {
53         int newPos;
54
55         if (parse(symbol, pos, newPos))
56         {
57             return match(rule, newPos, rulePos + 1, endPos);
58         }
59
60         return false;
61     }
62
63     // Terminal

```

```

64     if (pos < input.length() && input[pos] == symbol)
65     {
66         return match(rule, pos + 1, rulePos + 1, endPos);
67     }
68
69     return false;
70 }
71
72 int main()
73 {
74     int n;
75
76     cout << "Enter number of productions: ";
77     cin >> n;
78
79     cout << "Enter productions separately.\n";
80     cout << "Use # for epsilon.\n\n";
81
82     char startSymbol;
83
84     for (int i = 0; i < n; i++)
85     {
86         string p;
87
88         cout << "Production " << i + 1 << ": ";
89         cin >> p;
90
91         char lhs = p[0];
92         string rhs = p.substr(3);
93
94         grammar[lhs].push_back(rhs);
95
96         if (i == 0)
97             startSymbol = lhs;
98     }
99
100     cout << "\nEnter input string: ";
101     cin >> input;
102
103     int endPos = 0;
104
105     if (parse(startSymbol, 0, endPos) &&
106         endPos == input.length())
107     {
108         cout << "String is accepted";
109     }
110     else
111     {
112         cout << "String is not accepted";
113     }

```

```
114
115     return 0;
116 }
```

Listing 2: LEX/C++ program to Recursive Descent Parsing.

Output



```
Problems  Output  Debug Console  Terminal  Ports

Active code page: 65001

D:\code2026\cse324>cd "d:\code2026\cse324\lab-07-recursive-descent-parsing\" && g++ recursive-desc
recursive-descent-parsing\recursive-descent-parsing-all-production
Enter number of productions: 8
Use # for epsilon.

Production 1: E->TR
Production 2: R->+TR
Production 3: R->#
Production 4: T->FB
Production 5: B->*FB
Production 6: B->#
Production 7: F->(E)
Production 8: F->a

Enter input string: a+a*a
String is accepted
d:\code2026\cse324\lab-07-recursive-descent-parsing>
```

Figure 2: Output of Recursive Descent Parsing

3 Predictive parsing table

3.0.1 Question

1. Write a program to implement Predictive parsing table for the following grammar.

Write a C/ C++ program to implement Predictive parsing table for mini language.

Sample Input:

E -> eEx

E -> aBc

F -> fFy

G -> gGz

Sample Output:

	e	a	f	g
E	E->eEx	E->aBc		
F			F->fFy	
G				G->gGz

3.0.2 Solution

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include <map>
5  #include <set>
6  #include <cctype>
7  #include <iomanip>
8
9  using namespace std;
10
11 map<char, vector<string>> grammar;
12 map<char, set<char>> firstSet;
13 map<char, set<char>> followSet;
14 set<char> nonTerminals;
15 set<char> terminals;
16 char startSymbol;
17
18 void computeFirst(char c) {
19     if (!firstSet[c].empty()) return;
20
21     for (const string& prod : grammar[c]) {
22         if (prod == "#") {
23             firstSet[c].insert('#');
24             continue;
25         }
26
27         for (size_t i = 0; i < prod.length(); ++i) {
28             char nextChar = prod[i];
29
30             if (!isupper(nextChar)) {
31                 firstSet[c].insert(nextChar);
32                 break;
33             } else {
34                 computeFirst(nextChar);
35                 bool hasEpsilon = false;
36                 for (char f : firstSet[nextChar]) {
37                     if (f == '#') hasEpsilon = true;
38                     else firstSet[c].insert(f);
39                 }
40                 if (!hasEpsilon) break;
41                 if (i == prod.length() - 1) firstSet[c].insert('#');
42             }
43         }
44     }
45 }
46
47 void computeFollow() {
48     followSet[startSymbol].insert('$');
49
50     bool changed = true;

```

```

51 while (changed) {
52     changed = false;
53     for (char nt : nonTerminals) {
54         for (const string& prod : grammar[nt]) {
55             for (size_t i = 0; i < prod.length(); ++i) {
56                 char c = prod[i];
57                 if (isupper(c)) {
58                     size_t oldSize = followSet[c].size();
59
60                     if (i + 1 < prod.length()) {
61                         char nextChar = prod[i + 1];
62                         if (!isupper(nextChar)) {
63                             followSet[c].insert(nextChar);
64                         } else {
65                             bool hasEpsilon = false;
66                             for (char f : firstSet[nextChar]) {
67                                 if (f == '#') hasEpsilon = true;
68                                 else followSet[c].insert(f);
69                             }
70                             if (hasEpsilon) {
71                                 for (char f : followSet[nt])
72                                     ↪ followSet[c].insert(f);
73                             }
74                         } else {
75                             for (char f : followSet[nt]) followSet[c].insert(f);
76                         }
77
78                         if (followSet[c].size() != oldSize) changed = true;
79                     }
80                 }
81             }
82         }
83     }
84 }
85
86 int main() {
87     int numProds;
88     cout << "Enter number of productions: ";
89     cin >> numProds;
90
91     cout << "Enter productions:\n";
92     for (int i = 0; i < numProds; ++i) {
93         string prodLine;
94         cin >> prodLine;
95
96         char lhs = prodLine[0];
97         string rhs = prodLine.substr(3);
98
99         if (i == 0) startSymbol = lhs;

```



```

100
101     grammar[lhs].push_back(rhs);
102     nonTerminals.insert(lhs);
103
104     for (char ch : rhs) {
105         if (!isupper(ch) && ch != '#') terminals.insert(ch);
106     }
107 }
108 terminals.insert('$');
109
110 for (char nt : nonTerminals) computeFirst(nt);
111 computeFollow();
112
113 // First and Follow Output
114 cout << "\nFIRST AND FOLLOW SETS\n";
115 for (char nt : nonTerminals) {
116     cout << "FIRST(" << nt << ") = { ";
117     for (char c : firstSet[nt]) cout << c << " ";
118     cout << "}\n";
119
120     cout << "FOLLOW(" << nt << ") = { ";
121     for (char c : followSet[nt]) cout << c << " ";
122     cout << "}\n\n";
123 }
124
125 map<char, map<char, string>> parsingTable;
126 bool isLL1 = true;
127
128 for (char nt : nonTerminals) {
129     for (const string& prod : grammar[nt]) {
130         set<char> firstRHS;
131         if (prod == "#") {
132             firstRHS.insert('#');
133         } else {
134             for (size_t i = 0; i < prod.length(); ++i) {
135                 char nextChar = prod[i];
136                 if (!isupper(nextChar)) {
137                     firstRHS.insert(nextChar);
138                     break;
139                 } else {
140                     bool hasEpsilon = false;
141                     for (char f : firstSet[nextChar]) {
142                         if (f == '#') hasEpsilon = true;
143                         else firstRHS.insert(f);
144                     }
145                     if (!hasEpsilon) break;
146                     if (i == prod.length() - 1) firstRHS.insert('#');
147                 }
148             }
149         }
150     }
151 }

```

```

150
151     for (char t : firstRHS) {
152         if (t != '#') {
153             if (!parsingTable[nt][t].empty()) isLL1 = false;
154             parsingTable[nt][t] = prod;
155         } else {
156             for (char f : followSet[nt]) {
157                 if (!parsingTable[nt][f].empty()) isLL1 = false;
158                 parsingTable[nt][f] = "#";
159             }
160         }
161     }
162 }
163 }
164
165 // LL(1) Parsing Table Output
166 cout << "LL(1) PARSING TABLE\n";
167 cout << left << setw(8) << "NT";
168 for (char t : terminals) {
169     cout << setw(12) << t;
170 }
171 cout << "\n" << string(8 + terminals.size() * 12, '-') << "\n";
172
173 for (char nt : nonTerminals) {
174     cout << setw(8) << nt;
175     for (char t : terminals) {
176         string entry = "-";
177         if (!parsingTable[nt][t].empty()) {
178             entry = string(1, nt) + "->" + parsingTable[nt][t];
179         }
180         cout << setw(12) << entry;
181     }
182     cout << "\n";
183 }
184
185 if (!isLL1) {
186     cout << "\nGrammar is NOT LL(1) due to structural table conflicts!\n";
187 } else {
188     cout << "\nGrammar is successfully LL(1)!\n";
189 }
190 return 0;
191 }

```

Listing 3: LEX/C++ program to Predictive parsing table.

Output

Problems Output Debug Console **Terminal** Ports

Active code page: 65001

D:\code2026\cse324>cd "d:\code2026\cse324\lab-08-predictive-parsing\" && g++ predictive-parsing.cpp -o predictive-parsing

Enter number of productions: 4

Enter productions:

E->eEx

E->aBc

F->fFy

G->gGz

FIRST AND FOLLOW SETS

FIRST(E) = { a e }

FOLLOW(E) = { \$ x }

FIRST(F) = { f }

FOLLOW(F) = { y }

FIRST(G) = { g }

FOLLOW(G) = { z }

LL(1) PARSING TABLE

NT	\$	a	c	e	f	g	x	y	z
E	-	E->aBc	-	E->eEx	-	-	-	-	-
F	-	-	-	-	F->fFy	-	-	-	-
G	-	-	-	-	-	G->gGz	-	-	-

Grammar is successfully LL(1)!

d:\code2026\cse324\lab-08-predictive-parsing>

Figure 3: Output of Predictive parsing table

4 Remove single Line comment

4.0.1 Question

1. Write a C/C++ program to remove single Line comment from the input file (input.txt) source program..

4.0.2 Solution

```
1  /*Write a C/C Program to remove single line comment from the source program.
2  It reads charecter from the input.txt and write the result to output.txt file.*/
3
4  #include<iostream>
5  #include<fstream>
6  #include<string>
7  using namespace std;
8
9  int main()
10 {
11     //=====Write File=====
12     ofstream myFile("input.txt");
13
14     if(myFile.is_open())
15     {
16         myFile << "My name is Kaniz Fatema.\n";
17         myFile << "//Kaniz Fatema.\n";
18         myFile << "/*Kaniz Fatema.*\n";
19         myFile << "/*20245103154.*\n";
20         myFile << "Bangladesh is my country. //I live in Bangladesh.\n";
21
22         myFile.close();
23     }
24
25     //=====Read File=====
26
27     ifstream inFile("input.txt");
28
29     if(!inFile.is_open())
30     {
31         cout << "Error: input.txt file could not be opened!" << endl;
32         return 1;
33     }
34
35     //=====Output File=====
36     ofstream outFile("output.txt");
37
38     string line;
39
40     while(getline(inFile, line))
41     {
```

```

42     size_t pos = line.find("//");
43
44     if (pos != string::npos)
45     {
46
47         string cleanLine = line.substr(0, pos);
48         outFile << cleanLine << endl;
49     }
50     else
51     {
52         outFile << line << endl;
53     }
54 }
55
56
57 inFile.close();
58 outFile.close();
59
60 cout << "Single-line comments removed successfully!" << endl;
61
62 return 0;
63 }

```

Listing 4: LEX/C++ program to remove single Line comment.

Input File



```

lab-09-remove-single line-comments > input.txt
1  My name is Kaniz Fatema.
2  //Kaniz Fatema.
3  /*Kaniz Fatema.*/
4  /*20245103154.*/
5  Bangladesh is my country. //I live in Bangladesh.
6

```

Figure 4: Output of remove single Line comment

Output File



```
lab-09-remove-single line-comments > output.txt
1  My name is Kaniz Fatema.
2
3  /*Kaniz Fatema.*/
4  /*20245103154.*/
5  Bangladesh is my country.
6
```

Figure 5: Output of remove single Line comment

5 Three address code

5.0.1 Question

1. Write a C/C++ program to implement three address code

5.0.2 Solution

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4
5 using namespace std;
6
7 int main() {
8     string expr = "a+b-c*d";
9     int tempCount = 1;
10
11     cout << "Input Expression: " << expr << endl;
12     cout << "\nThree Address Code:" << endl;
13
14     string arg1 = string(1, expr[0]);
15
16     for (size_t i = 1; i < expr.length(); i += 2) {
17         char op = expr[i];
18         string arg2 = string(1, expr[i + 1]);
```

```

19     string res = "t" + to_string(tempCount++);
20
21     cout << res << " = " << arg1 << " " << op << " " << arg2 << endl;
22
23     arg1 = res;
24 }
25
26 return 0;
27 }

```

Listing 5: C/C++ program toThree address code.

Output

```

Problems  Output  Debug Console  Terminal  Ports

Active code page: 65001

D:\code2026\cse324>cd "d:\code2026\cse324\lab-10-three-address-code\" && g++ three-address-code.cpp
Input Expression: a+b-c*d

Three Address Code:
t1 = a + b
t2 = t1 - c
t3 = t2 * d

d:\code2026\cse324\lab-10-three-address-code>

```

Figure 6: Output of Three address code program

1 2

¹Kaniz Fatema

²ID: 20245103154